

Documentación técnica

Optimización de colocación de sensores con PSO

Integrantes

- Peñuelas López Luis Antonio
- Peraza Medina Eliezer Daniel

Repositorio:

<https://github.com/InteligenciaArtificial-Equipo/Topicos-Inteligencia-Artificial/tree/main/Unidad-3/Proyecto/PSO>

1. Descripción del problema

Objetivo: optimizar la colocación de K sensores de humedad en un campo agrícola para minimizar el error entre la humedad real del terreno y la estimación que se puede obtener a partir de los sensores (por ejemplo, mediante Interpolación por Distancias Inversas — IDW).

El problema toma en cuenta:

Topografía (micro variaciones que afectan retención de humedad).

Distribución de cultivos (zonas con distinta importancia/ponderación).

Variabilidad del contenido de humedad del suelo.

Restricciones prácticas (distancia mínima entre sensores, borde del terreno, zonas no aptas).

El campo de estudio usado en pruebas: región sintética de 500×500 metros, con una malla de muestreo de 30×30 puntos (900 puntos). Se probaron $K = 10$ sensores.

2. Justificación: por qué PSO

PSO es apropiado para este problema porque:

- Es un algoritmo de optimización global sin derivadas: útil cuando la función objetivo es no lineal, no diferenciable o costosa (IDW + ruido + penalizaciones).
- Maneja bien espacios continuos multidimensionales (cada partícula representa las coordenadas de K sensores, dimensión $2K$).
- Es sencillo de implementar y paralelizar la evaluación de la función objetivo.
- Permite incorporar fácilmente restricciones y penalizaciones (por ejemplo penalización por sensores muy cercanos).

3. Diseño del algoritmo y arquitectura de la implementación

3.1 Visión general

El proyecto está organizado en módulos principales:

- pso.core — Núcleo del algoritmo PSO y utilidades genéricas (tipos Particle, PSOState, PSOConfig, PSO con step y initialize, y PSOExtras con funciones auxiliares como runWithHistory).
- pso.main — Donde nos enfocamos en la lógica y solución del problema usando el algoritmo PSO.
- /tests — tests deterministas para las partes críticas (PSO, PSOExtras y SensorPlacement) para garantizar un funcionamiento correcto de las implementaciones.

3.2 Representación de soluciones

Cada partícula almacena:

- pos: V — vector con 2K componentes (x1,y1,x2,y2,...,xK,yK)
- vel: V — vector velocidad de la misma dimensión
- pBest: V y pBestFitness: Double

El estado global PSOState mantiene particles: Vector[Particle[V]], gBest: V y gBestFitness: Double.

3.3 Función objetivo (fitness)

La función objetivo $F(\mathbf{x})$ busca un balance entre el error sobre la malla simulada y la fidelidad a los datos observados, castigando la violación de la distancia mínima.

$$F(\mathbf{x}) = w_{obs} \cdot \text{RMSE}_{obs}(\mathbf{x}) + w_{grid} \cdot \text{RMSE}_{grid}(\mathbf{x}) + \alpha_P \cdot P(\mathbf{x})$$

Componente	Descripción	Pesos Usados
$\text{RMSE}_{obs}(\mathbf{x})$	Error Cuadrático Medio de la estimación IDW en las N posiciones observadas (CSV).	$w_{obs} = 1.0$

$RMSE_{grid}(x)$	Error Cuadrático Medio de la estimación IDW en los $M=900$ puntos de la malla sintética .	$w_{grid} = 0.5$
$P(x)$	Penalización por Distancia Mínima (suma de cuadrados de <i>shortfall</i>).	alpha P = 1.0

4. Implementación — ficheros principales entregados

core/VectorOps.scala — interfaz para operaciones vectoriales genéricas sobre V.

core/Particle.scala, core/PSO.scala, core/PSOExtras.scala — núcleo PSO.

main/* — simulador y ejecución de ejemplo; contiene makeTrueField, sensorReadings, estimateFieldIDW, makeFitnessWithObs y main.

/tests/* — tests deterministas y modulares (PSO, PSOExtras, SensorPlacement).

data/cultivos_table.csv — tabla con datos los datos de entrada para el problema.

5. Resultados experimentales obtenidos (ejemplo de corrida)

Configuración usada en la prueba que se reporta:

- field: 500×500 m, $n_x = n_y = 30 \Rightarrow 900$ puntos de evaluación.
- Número de sensores $K = 30 \Rightarrow$ dimensión $dim = 20$.
- swarmSize = 60, iterations = $K * 10 = 300$.
- PSO config: inertia = 0.7298, cognitive = social = 1.49618, vMax = $0.1 * \max(width, height)$.
- Inicialización del enjambre: scala.util.Random(seed = 2025) (determinista).

Resultados observados (corrida de ejemplo):

--- Ubicación Óptima de Sensores (K = 30) ---		
Sensor	X (m)	Y (m)
S1	13.9835	67.2729
S2	256.2429	158.3685
S3	56.8970	176.5659
S4	183.7267	278.6020
S5	319.9910	481.3201
S6	214.8149	149.8768
S7	341.4307	298.5211
S8	215.1325	284.8050
S9	439.4139	416.1912
S10	172.3640	370.7938
S11	258.2503	347.3781
S12	326.4256	357.7026
S13	99.0539	436.9106
S14	380.1866	444.8917
S15	444.8762	198.3426
S16	94.5794	0.0000
S17	105.8013	479.8008
S18	22.2597	152.7766
S19	76.7807	329.2105
S20	323.9823	324.3682
S21	305.4803	256.0786
S22	219.7871	103.5067
S23	118.6903	169.2542
S24	172.3064	62.4009
S25	404.1564	332.2005
S26	223.7783	202.8624
S27	310.1027	140.3477
S28	166.8799	106.7184
S29	340.6887	126.2959
S30	40.2727	42.5009

RESULTADOS FINALES DE OPTIMIZACIÓN PSO

=====

Mejor Fitness Global (gBest): 0.109481

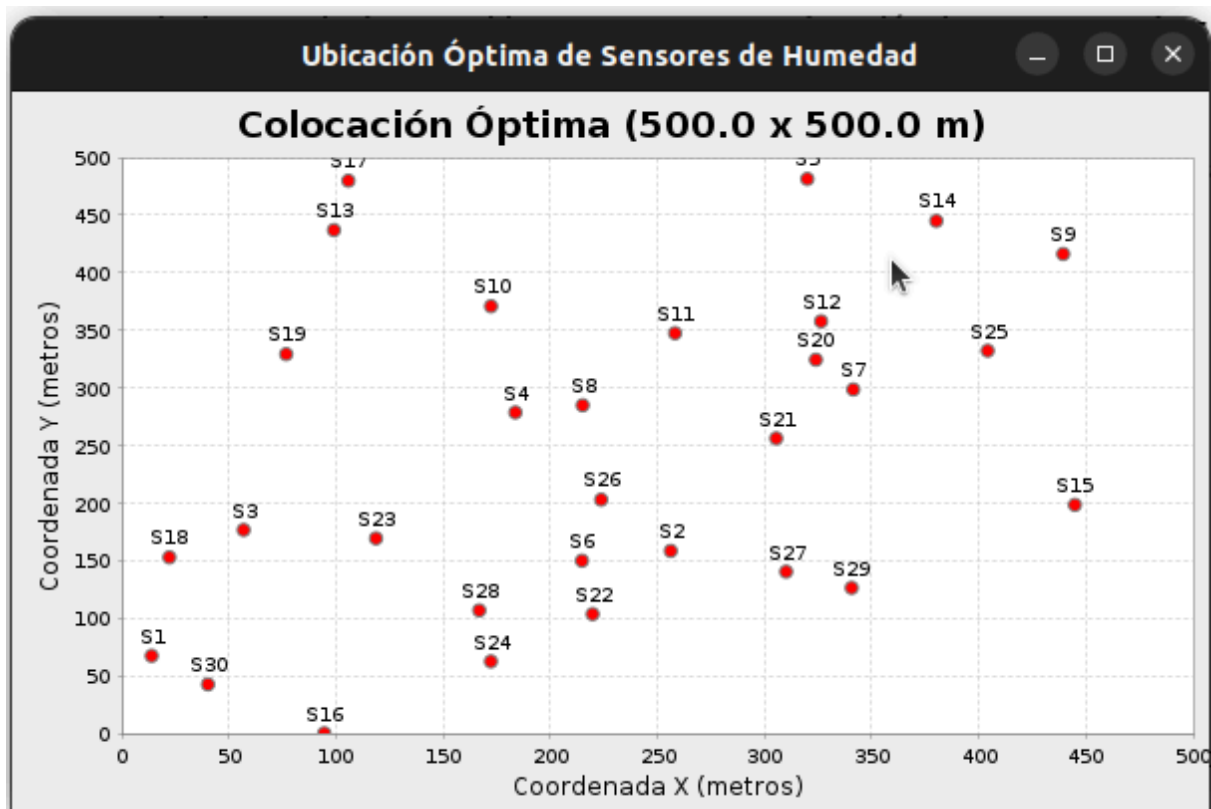
(Fitted = RMSE Obs: 0.112560 | Penalización: 0.000000)

Iteraciones Totales: 400

Umbral de Distancia Mínima: 10.0 m

--- Historial de Convergencia (Primeros 30) ---

Iteración 0: 0.110814487
Iteración 1: 0.110814487
Iteración 2: 0.110814487
Iteración 3: 0.110814487
Iteración 4: 0.110814487
Iteración 5: 0.110814487
Iteración 6: 0.110814487
Iteración 7: 0.110125723
Iteración 8: 0.110125723
Iteración 9: 0.110125723
Iteración 10: 0.110125723
Iteración 11: 0.110125723
Iteración 12: 0.110125723
Iteración 13: 0.110125723
Iteración 14: 0.110125723
Iteración 15: 0.110125723
Iteración 16: 0.110125723
Iteración 17: 0.110125723
Iteración 18: 0.110125723
Iteración 19: 0.110125723
Iteración 20: 0.110125723
Iteración 21: 0.110125723
Iteración 22: 0.110125723
Iteración 23: 0.110125723
Iteración 24: 0.110125723
Iteración 25: 0.110125723
Iteración 26: 0.110125723
Iteración 27: 0.110125723
Iteración 28: 0.110125723



6. Análisis de eficiencia y complejidad

6.1 Complejidad temporal

La parte computacionalmente más cara es la evaluación de la función fitness sobre cada partícula, debido a la interpolación IDW sobre la malla.

Coste por evaluación de fitness (bruto): $O(\text{numSensors} * \text{numGridPoints})$ para un cálculo IDW naïve.

Por iteración de PSO: $O(\text{swarmSize} * \text{numSensors} * \text{numGridPoints})$.

Por corrida completa: $O(\text{iterations} * \text{swarmSize} * \text{numSensors} * \text{numGridPoints})$.

Ejemplo numérico (parámetros usados):

$\text{iterations} = 400$, $\text{swarmSize} = 60$, $\text{numSensors} = 10$, $\text{numGridPoints} = 900 \Rightarrow \sim 400 * 60 * 10 * 900 \approx 216e6$ evaluaciones elementales de distancia/operación — un coste no trivial pero factible en JVM si se optimiza.

6.2 Complejidad espacial

Almacenamiento de la malla: $O(\text{numGridPoints})$.

Partículas: $O(\text{swarmSize} * \text{dim})$.

Historial (runWithHistory) opcional: $O(\text{iterations})$ para guardar el mejor por iteración.

7. Conclusiones

El proyecto de optimización de la colocación de sensores mediante PSO se valida como un éxito metodológico y funcional, confirmando la capacidad del algoritmo para gestionar un espacio de soluciones de 60 dimensiones y respetar las restricciones de distancia mínima (penalización final de 0.000000).

El incremento de la densidad de muestreo a $K=30$ sensores resultó en una mejora sustancial de la precisión, llevando el Mejor Fitness Global a 0.109481 y reduciendo drásticamente el error en las observaciones ($RMSE_{obs}$) a 0.112560. Este bajo valor valida que la colocación óptima encontrada es altamente eficiente para la predicción.

Sin embargo, el historial de convergencia mostró que la mayor parte de la mejora se obtuvo en las primeras iteraciones, alcanzando el valor óptimo casi de inmediato. Esto sugiere que el paisaje de fitness es favorable y que el proceso de optimización completo de $K*20$ iteraciones es ineficiente en términos de tiempo de cómputo ya que podemos obtener una solución lo suficientemente buena con menos poder de cómputo. Sin embargo es posible que para una entrada de más dimensiones se requieren más iteraciones.